



CLIENTGUARD

ClientGuard v8.0.0

Complete Operator Manual



Defensive Cybersecurity Assessment Platform
for Authorized Networks



Current Stable Release



Written by **Raid9sec**



Scoring Model
7.5.0-1



Report Schema
6



Storage Schema
3

FIELD MANUAL

Built for installed ClientGuard operations

This manual begins after installation and uses the managed CURRENT runtime for assisted scans, direct CLI assessments, endpoints, servers, network infrastructure, websites, reporting, retesting, and native v8 workflows.

RELEASE IDENTITY

ClientGuard 8.0.0

Scoring 7.5.0-1 | Report Schema 6 |
Storage Schema 3

MANUAL NAVIGATION

Table of Contents

Use the links below in the HTML manual. In the PDF, chapter numbering mirrors the operational flow from readiness through reporting, retesting, and platform-specific scenarios.

START Release identity and installed paths p. 2	01 Operating model, authorization, and what ClientGuard does p. 4	02 Where ClientGuard runs and what it can assess p. 7
03 First-run readiness and dependency checks p. 9	04 Three ways to operate ClientGuard p. 12	05 Interface, routing, scope, CIDR, and exclusions p. 14
06 Assistant/menu scan - start to finish p. 16	07 Direct CLI assessments and profile selection p. 19	08 Ethernet assessment scenarios p. 21
09 Connected Wi-Fi assessment scenarios p. 23	10 Website and web-application assessment scenarios p. 25	11 Windows endpoint and Windows Server scenarios p. 27
12 Linux server scenarios p. 29	13 Routers, firewalls, appliances, and network infrastructure p. 30	14 Mixed client network scenario + large enterprise scaling p. 31
15 Native v8 active discovery and controlled workflow JSON p. 37	16 What each external tool does p. 39	17 Command-line option reference - common --* flags p. 41
18 Reports, evidence, CVE/CVSS/EPSS/KEV, and confidence p. 45	19 Retest, compare, safe validation, and dispositions p. 47	20 Dashboard, presets, checkpoints, and operational utilities p. 49
21 Troubleshooting and common operator mistakes p. 51	22 Field checklists and mock terminal sessions p. 53	23 Apple/macOS endpoints, Mac servers, and other Apple devices p. 55
A Installed path reference p. 60	B OS / server coverage matrix p. 61	C Command recipes p. 63
END Final operator reminder p. 64		

Release identity and installed paths

This manual is bound to the ClientGuard v8.0.0 CURRENT STABLE release. The managed CURRENT runtime is the executable operators should normally call. The release and rollback copies are preserved for integrity and recovery, not for routine scanning.

Official download: <https://github.com/Raid9sec/ClientGuard>. See section 1.3 for download, verification, and first-launch commands.

Item	Current path / identity
Managed runtime	/home/kali/ClientGuard/current/clientguard.py
CURRENT STABLE SHA-256	11db71b93aa030493f70a36023da36012868c2e66b2c278854fbc1f2e6daf0f5
Release copy	/home/kali/ClientGuard/releases/8.0.0/clientguard.py
Stable release seal	/home/kali/ClientGuard/releases/8.0.0/STABLE_RELEASE_SEAL.json
Rollback runtime	/home/kali/ClientGuard/rollback/clientguard_before_8.0.0.py
Rollback SHA-256	f0413238bc7c36c7b0dd6f1f09dfae59a24d37d9f911e4fdcc853e9005cd43c
Release archive	/home/kali/ClientGuard_release_archive/v8.0.0/
Archive manifest SHA-256	0589aff2720f35d6f4b867a018023806b5e7b9da10e87093fdcbb86e825b7de3
Typical report root	/home/kali/reports/<client>/<year>/<Profile>/<run>/

Verify the installed runtime

CLIENTGUARD TERMINAL

```
$ sha256sum /home/kali/ClientGuard/current/clientguard.py
11db71b93aa030493f70a36023da36012868c2e66b2c278854fbc1f2e6daf0f5
/home/kali/ClientGuard/current/clientguard.py
$ python3 /home/kali/ClientGuard/current/clientguard.py --version
8.0.0
```

**NOTE: Report location**

Interactive assessments print a Saved: line at completion. Use that line as the authoritative location because the report root can be changed by configuration. The examples in this manual use `/home/kali/reports`.

Operating model, authorization, and what ClientGuard does

ClientGuard is a defensive assessment platform for systems and networks you own or are explicitly authorized to assess. It combines network discovery, service detection, conservative vulnerability intelligence, safe validation, evidence handling, and client-ready reporting without turning a correlation into an exploit or an automatic business decision.

1.1 The 10-stage v8 assessment model

Stage	Purpose	Operator interpretation
1 Preflight	Check runtime, tools, interfaces, storage, and basic readiness.	A failed preflight is a reason to stop or repair readiness before scanning.
2 Authorization	Bind operator, client, target, scope, and exclusions.	The scan must remain inside this boundary.
3 Asset discovery	Find reachable authorized systems when discovery is appropriate.	A known single /32 target may skip separate host discovery.
4 Service detection	Identify reachable ports, protocols, products, and conservative version evidence.	Open means exposed, not vulnerable.
5 Vulnerability correlation	Compare sufficient product/version/CPE evidence to CVE intelligence.	Correlation is not proof.
6 Safe validation	Collect additional non-destructive evidence where supported.	Validation can increase confidence but does not close or remediate.
7 Evidence processing	Preserve observations and receipts.	Evidence should remain attributable and reproducible.
8 Risk intelligence	Add CVSS, EPSS, KEV, and prioritization context.	Risk intelligence assists triage; it does not replace analyst judgment.
9 Reporting	Produce HTML/JSON and related artifacts.	Read the evidence and confidence, not only the severity label.
10 Governance readiness	Support explicit analyst lifecycle actions.	READY never means CLOSED.

1.2 Non-negotiable interpretation rules

- An open port is exposure evidence, not automatic vulnerability proof.
- A CVE correlation is a candidate relationship, not exploit confirmation.

- No CVE match can be the correct result when version/CPE evidence is too generic.
- Not detected does not mean fixed.
- Accepted risk does not reduce technical exposure.
- ClientGuard does not automatically exploit, remediate, close, reopen, accept risk, expand scope, persist credentials, or silently upload/export evidence.
- Use safe web validation defaults. Do not add destructive SQL injection, file-write, OS-shell, password-attack, persistence, or denial-of-service options to routine assessments.



CAUTION: Authorization is a technical input, not a checkbox

Document the client, authorized targets, exclusions, maintenance windows, prohibited actions, and escalation contact before beginning a real engagement. If scope is uncertain, stop and clarify it.

1.3 Official download and first launch

ClientGuard is published from the official Raid9sec GitHub repository. Use the release asset when you only need the executable script, or clone the repository when you also want the documentation and repository files. If ClientGuard is already installed under `/home/kali/ClientGuard`, keep using the managed `/home/kali/ClientGuard/current/clientguard.py` and do not manually overwrite it.

Official GitHub repository

<https://github.com/Raid9sec/ClientGuard>

ClientGuard v8.0.0 release

GitHub Releases - downloadable assets and checksums

Method A - download the standalone v8.0.0 script

DOWNLOAD FROM GITHUB RELEASE

```
$ cd ~/Downloads
$ curl -L -o clientguard.py \
  https://github.com/Raid9sec/ClientGuard/releases/download/v8.0.0/clientguard.py
$ sha256sum clientguard.py
11db71b93aa030493f70a36023da36012868c2e66b2c278854fbc1f2e6daf0f5  clientguard.py
$ chmod +x clientguard.py
$ python3 clientguard.py --version
8.0.0
$ python3 clientguard.py --v8-self-test
```

Method B - clone the public repository

```
$ cd ~/Downloads
$ git clone https://github.com/Raid9sec/ClientGuard.git
$ cd ClientGuard
$ sha256sum clientguard.py
```

Where ClientGuard runs and what it can assess

2.1 Operator platform

ClientGuard v8.0.0 is intended to run from Kali Linux or a compatible Linux assessment workstation where its supporting utilities are available. The target does not need ClientGuard installed. ClientGuard reaches authorized targets over the network and evaluates the services those targets expose.

Normal installed invocation

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--assistant
```

2.2 Target coverage

Target class	Examples	What ClientGuard can observe	Notes
Windows endpoints	Windows 10/11 Pro or Enterprise	RPC, SMB, RDP, WinRM, IIS/web services, SQL services, other listening TCP services.	Host firewall may intentionally filter services. Version evidence determines whether CVE correlation is possible.
Windows Server	Server 2019/2022/2025, member servers, domain controllers	AD-related services, DNS, Kerberos, LDAP/LDAPS, SMB, RDP, WinRM, IIS, databases and custom services.	Use exact scope and maintenance-window awareness; do not treat exposed AD services as vulnerabilities by themselves.
Linux servers	Debian, Ubuntu, RHEL-family, appliances	SSH, HTTP/S, DNS, mail, databases, RPC/NFS and application services.	Package backports can make banner versions misleading; analyst validation matters.
Web servers / apps	Apache, Nginx, IIS, reverse proxies, portals	HTTP/S headers, TLS, web fingerprints, Nikto/Nuclei where appropriate, conservative SQLMap checks on explicitly authorized URLs.	Web profile is URL-oriented; do not run SQLMap against pages without parameters or authorization.

Target class	Examples	What ClientGuard can observe	Notes
Routers / firewalls	SOHO routers, enterprise gateways, network appliances	Management ports, web interfaces, SSH/Telnet exposure, TLS posture and service versions.	Router profile is for defensive posture, not credential attacks.
Connected Wi-Fi networks	Corporate or lab WLAN after legitimate association	Same IP-layer assessment as Ethernet through wlan0.	ClientGuard is not a Wi-Fi cracking/deauthentication suite.
BSD / other appliances	Reachable networked BSD hosts and service appliances	Generic service detection and web/TLS observations.	Coverage is service-driven; platform-specific remediation may require analyst context.
Apple/macOS systems	MacBook, iMac, Mac mini, Mac Studio, Mac Pro, Apple Silicon or Intel Macs	Service-driven network observations such as SSH/Remote Login, SMB, Screen Sharing/VNC, HTTP/S, printing and third-party services when reachable.	The target does not need ClientGuard installed. Coverage is network/service driven; local FileVault, Gatekeeper, SIP, MDM and TCC posture require a separate authenticated/local audit.

Apple coverage note: Macs are assessed the same way as other network targets - by authorized reachability and exposed services. Apple Silicon and Intel Macs are both supported as network targets. iPhone/iPad/Apple TV coverage is limited to reachable services and does not constitute a local mobile-device or MDM compliance audit.

2.3 What ClientGuard is not

- It is not an EDR agent and does not need to be installed on every target.
- It is not a credential-dumping, password-cracking, exploit-framework, or persistence tool.
- It does not prove patch status solely from an open port or generic service banner.
- It does not replace authenticated configuration auditing when a client requires CIS/STIG policy checks inside the operating system.

3.1 Confirm the managed runtime

Command	<code>python3 /home/kali/ClientGuard/current/clientguard.py --version</code>
What it does	Confirms the installed ClientGuard release identity.
Use it when	At the beginning of an engagement and after upgrades.
What to expect	v8.0.0 should print 8.0.0.

Command	<code>python3 /home/kali/ClientGuard/current/clientguard.py --v8-self-test</code>
What it does	Runs the native v8 local self-test and reports frozen model/schema identities and safety flags.
Use it when	Before a significant engagement or after host changes.
What to expect	JSON containing self_test: PASS, stage_count: 10, Scoring Model 7.5.0-1, Report Schema 6, Storage Schema 3, and false for exploit_execution/production_write/storage_migration.

Command	<code>python3 /home/kali/ClientGuard/current/clientguard.py --v8-capabilities</code>
What it does	Lists scanner/tool capabilities visible to v8.
Use it when	When a scanner appears unavailable or before a web assessment.
What to expect	Entries such as nmap, nikto, nuclei, sqlmap, openssl with AVAILABLE/unavailable state.

3.2 Compatibility precheck

Command	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --precheck</code>
What it does	Runs the established environment/tool readiness check without starting the client assessment.
Use it when	Before the first scan on a new Kali workstation.
What to expect	A readiness summary. Fix required tool failures before a full assessment.

3.3 Tool installation planning

Command	<code>python3 /home/kali/ClientGuard/current/clientguard.py --install-tools --dry-run</code>
What it does	Shows what the dependency installer would change without installing it.
Use it when	When preparing a workstation and you want to review package changes first.
What to expect	A plan only; no assessment is started.

Command	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --install-tools</code>
What it does	Installs normal supported tool dependencies.
Use it when	After reviewing the dry run and when you control the assessment workstation.
What to expect	Package/tool installation output. Re-run --precheck afterward.

Command	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --install-tools --include-optional</code>
What it does	Adds optional supported scanners to the tool installation workflow.
Use it when	Only when those scanners are part of your approved methodology.

Command	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --install-tools --include-optional</code>
What to expect	Additional packages may be installed; capability inventory should update afterward.
Caution	Do not install or enable tools merely because they exist. Match tools to engagement authorization.

4.1 Mode A - Assistant/menu driven

Launch the Command Assistant

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--assistant
```

Mock main menu

CLIENTGUARD TERMINAL

```
CLIENTGUARD COMMAND ASSISTANT  
Choose a task:  
1) Standard Client Audit  
2) Quick Network Audit  
3) Wi-Fi Audit  
4) Router Audit  
5) Website Audit  
6) Retest / Compare Previous Client  
7) Verify One Finding  
8) Use Saved Preset  
9) Show Recent Commands  
10) Generate Bash/Zsh Completion  
11) Safe Validate a Previous Finding  
12) Explicit Nuclei URL Validation  
13) Mark Finding Disposition  
0) Back  
Select:
```

Use the assistant when you want ClientGuard to prompt for interface, client name, assessor, scope, authorization, and the action to run. It is the best starting point for operators who want a guided workflow and a generated command they can review before execution.

4.2 Mode B - Direct profile CLI

Direct Standard assessment

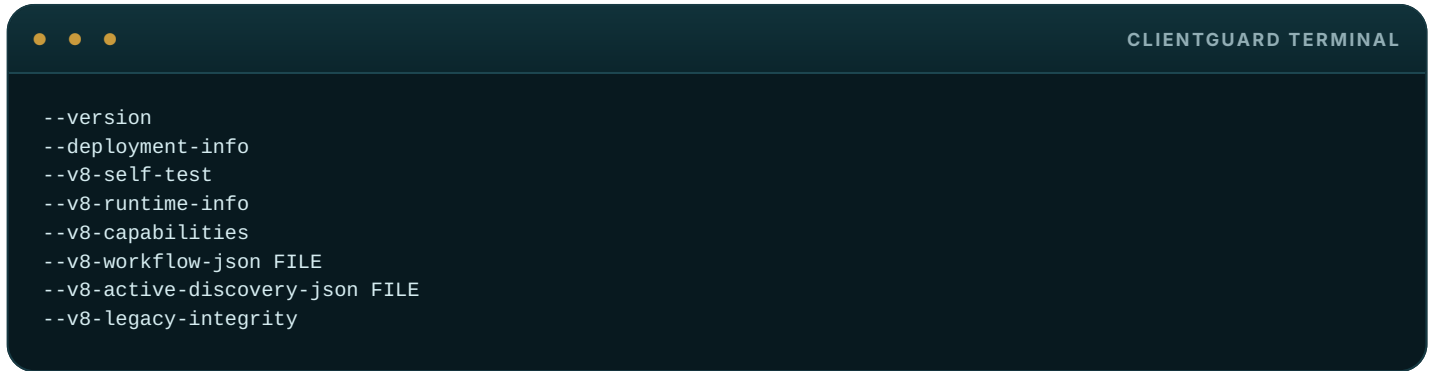
CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--profile standard --target 10.40.20.15/32 --interface eth0  
--client-name "Acme-Internal" --assessor "your name"
```

Direct mode is useful for repeatable operations, runbooks, and known scopes. The assistant ultimately produces commands in this style.

4.3 Mode C - Native v8 JSON controls

Native v8 commands

A terminal window with a dark blue background and a title bar that says "CLIENTGUARD TERMINAL". The terminal displays a list of command-line options for the native v8 JSON interface. The options are listed in a monospaced font, with some options followed by "FILE" to indicate they take a file path as an argument.

```
--version
--deployment-info
--v8-self-test
--v8-runtime-info
--v8-capabilities
--v8-workflow-json FILE
--v8-active-discovery-json FILE
--v8-legacy-integrity
```

Native v8 JSON modes are explicit, receipt-oriented interfaces used when you want machine-readable authorization and scope inputs. The compatibility runtime continues to provide the established profiles, menus, reports, retests, and governance workflows.

5.1 Identify the interface

Ethernet and Wi-Fi visibility

CLIENTGUARD TERMINAL

```
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 10.40.20.44/24
wlan0 DOWN
$ ip route
10.40.20.0/24 dev eth0 proto kernel scope link src 10.40.20.44
```

Choose the interface whose route actually reaches the authorized target. eth0 is commonly Ethernet; wlan0 is commonly a Wi-Fi adapter. Docker bridges, veth devices, loopback, and unrelated VPN interfaces should not be selected unless the engagement explicitly uses them.

5.2 Scope sizes

Scope	Meaning	Typical use
10.40.20.15/32	Exactly one IPv4 host.	Safest for a single server, endpoint, router, or appliance.
10.40.20.0/24	Addresses 10.40.20.0 through 10.40.20.255.	Authorized subnet assessment; verify exclusions.
scope file	A list/ruleset supplied separately.	Larger engagements where exact approved ranges are easier to maintain in a file.
--exclude 10.40.20.50/32	Remove a target from an otherwise authorized range.	Sensitive equipment, printers, life-safety devices, or client-prohibited hosts.



CAUTION: Do not accept the suggested /24 automatically

The assistant may suggest the local subnet. Replace it with the written client scope. A single server engagement should normally use /32, not the entire LAN.

5.3 Scope file example

client_scope.txt (illustrative)

```
10.40.20.0/24
10.40.30.10/32
# Exclude sensitive device separately with --exclude 10.40.20.50/32
```

Full profile using a scope file

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile full --scope-file client_scope.txt --interface eth0 --exclude
10.40.20.50/32 --client-name "Acme-Q3" --assessor "your name"
```


Assistant/menu scan - start to finish

This is the recommended guided workflow for a normal client assessment. The scenario below is a real-world style internal server review: a security consultant is connected by Ethernet to an authorized client VLAN and is allowed to assess one Windows server at 10.40.20.15.

6.1 Start the assistant

Terminal

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--assistant
```

6.2 Choose Standard Client Audit

Menu selection

CLIENTGUARD TERMINAL

```
Select: 1
```

6.3 Choose the correct interface

Mock prompt

CLIENTGUARD TERMINAL

```
Detected interfaces:  
[1] lo UNKNOWN 127.0.0.1/8  
[2] eth0 UP 10.40.20.44/24 10.40.20.0/24  
[3] wlan0 DOWN  
Select interface [1]: 2
```

Why: eth0 owns the source address and route to 10.40.20.0/24. Selecting loopback, Docker, or an unrelated VPN interface would produce misleading reachability results.

6.4 Enter client/assessor and exact scope

Mock prompt

```
Client/company name: Acme-Internal
Assessor: your name
Authorized scope [10.40.20.0/24]: 10.40.20.15/32
```

The /32 ensures the generated assessment can target only the approved Windows server, not every device on the client VLAN.

6.5 Review the generated command

Mock generated command

```
sudo python3 clientguard_v7_12_0.py --profile standard --target
10.40.20.15/32 --interface eth0 --client-name Acme-Internal --assessor "your name"
```

The assistant may display the embedded compatibility-runtime filename in its generated-command preview. When you launch through the installed v8 wrapper, the compatibility behavior is delegated internally; continue invoking the managed v8 path for normal operations.

6.6 Run and confirm authorization

Mock choice

```
1) Run now
2) Save as preset
3) Show command only
4) Edit/rebuild
0) Cancel
Choose: 1
Confirm you are authorized to assess this target and run now? [y/N]:
y
```

6.7 What to expect during progress

Mock progress

```
Preflight COMPLETE
Host Discovery COMPLETE or SKIPPED for known /32
Service Detection COMPLETE
CVE / EPSS / KEV COMPLETE
Safe Validation COMPLETE
Report Generation COMPLETE
Complete COMPLETE | Overall 100.00%
```

A single known target can legitimately show Host Discovery SKIPPED because service detection can proceed directly. A CVE count of zero is also valid when detected services do not provide reliable product/version/CPE evidence.

6.8 Find and open the report

Mock completion

```
Saved: Standard/4_Aug242026_1015am
report.html - report.json
```

Open from the observed report root

```
$ find /home/kali/reports -type d -name '4_Aug242026_1015am'
2>/dev/null
/home/kali/reports/Acme-Internal/2026/Standard/4_Aug242026_1015am
$ xdg-open
/home/kali/reports/Acme-Internal/2026/Standard/4_Aug242026_1015am/report.html
```

Direct CLI assessments and profile selection

Profile / mode	Use when	Typical behavior	Example
quick	You need a fast exposure snapshot over an authorized range.	Faster discovery/service review; less depth than standard/full.	--profile quick --target 10.40.20.0/24 --interface eth0
standard	Normal client endpoint/server assessment.	Balanced service detection, intelligence, safe validation, and reporting.	--profile standard --target 10.40.20.15/32 --interface eth0
full	Broader/deeper authorized assessment or baseline/retest workflow.	More comprehensive workflow and supporting evidence where enabled.	--profile full --target 10.40.20.0/24 --interface eth0
deep	Deep single-host assessment where additional time is approved.	Longer service and validation workflow; use maintenance-window awareness.	--profile deep --target 10.40.20.25 --interface eth0
router	Gateway/network appliance posture review.	Targets gateway/service exposure and relevant web/TLS posture.	--profile router --target 10.40.20.1 --interface eth0
web	Website or web application URL.	HTTP/S-focused checks, optional authorized web validators.	--profile web --url https://portal.example.com

7.1 Direct Standard server assessment

Command

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.40.20.15/32 --interface eth0
--client-name "Acme-Internal" --assessor "your name"
```

7.2 Full authorized subnet with exclusion

Command

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--profile full --target 10.40.20.0/24 --exclude 10.40.20.50/32  
--interface eth0 --client-name "Acme-Q3" --assessor "your name"
```

7.3 Deep single-host assessment

Command

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--profile deep --target 10.40.20.25 --interface eth0 --client-name  
"Acme-AppServer" --assessor "your name"
```



NOTE: Quick vs Standard vs Full/Deep

Choose the least intrusive profile that answers the client question. More depth is not automatically better. A deep scan can take longer and may exercise more application behavior even when it remains non-exploitative.

8.1 Scenario A - one employee workstation

Situation: The client authorizes a single managed endpoint at 10.40.20.88. The goal is to document externally reachable services from the assessment VLAN.

Readiness

CLIENTGUARD TERMINAL

```
$ ip -br addr show eth0
eth0 UP 10.40.20.44/24
$ ip route get 10.40.20.88
10.40.20.88 dev eth0 src 10.40.20.44
```

Assessment

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.40.20.88/32 --interface eth0
--client-name "Acme-Endpoint" --assessor "your name"
```

Expect: possibly many filtered ports due to endpoint firewall, plus any intentionally exposed services such as RDP, SMB, local web management, or third-party applications. Filtered is meaningful posture evidence; do not disable the client firewall just to create findings.

8.2 Scenario B - authorized server VLAN

Situation: The client authorizes 10.40.30.0/24 but excludes a legacy manufacturing controller at .50.

Assessment

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile full --target 10.40.30.0/24 --exclude 10.40.30.50/32
--interface eth0 --client-name "Acme-ServerVLAN" --assessor "your name"
```

Expect: asset discovery across the approved subnet, service detection per reachable host, CVE correlation only where reliable evidence exists, and one client report tree. Verify the exclusion appears in the effective scope before continuing.

8.3 Scenario C - known firewall-filtered host

Manual reachability context

CLIENTGUARD TERMINAL

```
$ nmap -Pn 10.40.20.88
Host is up.
Not shown: 1000 filtered tcp ports
```

Interpretation: the host is reachable at layer 2/3 but the firewall is dropping probes. This is not a scan failure. ClientGuard should report the observed filtering posture and avoid inventing service findings.

Connected Wi-Fi assessment scenarios

ClientGuard assesses IP networks reached through a wireless interface after legitimate association. It does not need monitor mode and is not intended for Wi-Fi password cracking, deauthentication, handshake capture, or bypassing network access controls.

9.1 Confirm legitimate connection

Wireless context

CLIENTGUARD TERMINAL

```
$ nmcli device status
DEVICE TYPE STATE CONNECTION
wlan0 wifi connected Corp-Guest-Audit
eth0 ethernet disconnected --
$ ip -br addr show wlan0
wlan0 UP 10.30.50.24/24
$ ip route
10.30.50.0/24 dev wlan0 proto kernel scope link src 10.30.50.24
```

9.2 Assistant Wi-Fi audit

Launch

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--assistant
```

Mock selection

CLIENTGUARD TERMINAL

```
Choose a task: 3) Wi-Fi Audit
Select interface: wlan0
Client/company name: Acme-WLAN
Authorized scope: 10.30.50.0/24
Confirm authorization: y
```

9.3 Direct quick Wi-Fi assessment

Command


```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--profile quick --target 10.30.50.0/24 --interface wlan0 --client-name  
"Acme-WLAN" --assessor "your name"
```

Expect: the same IP-layer evidence you would collect over Ethernet - reachable hosts, exposed services, banners, CVE correlation, and reports. The medium is Wi-Fi, but the assessment remains a network/service assessment.

**CAUTION: Guest networks and isolation**

Client isolation can prevent one Wi-Fi station from reaching another. If ClientGuard sees only the gateway or no peers, that may be intentional WLAN segmentation, not a tool failure.

Website and web-application assessment scenarios

Use the web profile when the engagement is centered on an explicitly authorized HTTP/S URL. ClientGuard can combine HTTP/TLS observation with Nikto, Nuclei validation, and conservative SQLMap checks when those tools are available and appropriate.

10.1 Preflight the URL manually

Basic HTTP/TLS visibility

CLIENTGUARD TERMINAL

```
$ curl -I https://portal.example.com
HTTP/2 200
server: nginx
content-type: text/html
$ openssl s_client -connect portal.example.com:443 -servername
portal.example.com </dev/null
```

These commands establish that the URL resolves/reaches the service and help distinguish DNS/TLS problems from ClientGuard workflow issues. They do not replace the ClientGuard report.

10.2 Assistant Website Audit

Menu path

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--assistant
Select: 5 # Website Audit
Authorized URL: https://portal.example.com
Client/company name: Acme-Web
Confirm authorization: y
```

10.3 Direct web profile

Command

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile web --url https://portal.example.com --client-name "Acme-Web"
--assessor "your name"
```

10.4 Explicit Nuclei validation

Command	<code>python3 /home/kali/ClientGuard/current/clientguard.py --nuclei-validate https://portal.example.com</code>
What it does	Runs the ClientGuard-supported explicit Nuclei URL validation path.
Use it when	When a web target is explicitly authorized and you need template-based safe validation beyond normal profile evidence.
What to expect	Nuclei findings/evidence routed through ClientGuard semantics.
Caution	Templates vary. Review engagement limits and avoid intrusive templates outside authorization.

10.5 Conservative SQLMap check

Command	<code>python3 /home/kali/ClientGuard/current/clientguard.py --sqlmap-check 'https://portal.example.com/item?id=1'</code>
What it does	Runs the ClientGuard-supported conservative SQLMap check against an explicitly authorized parameterized URL.
Use it when	Only when the URL has a parameter and SQL-injection testing is approved.
What to expect	Evidence may report no injection, possible indicators, or a candidate needing analyst review.
Caution	Do not add <code>--os-shell</code> , file read/write, tamper, password attack, data dumping, or destructive options to a routine ClientGuard assessment.

10.6 What a clean web result means

A clean Nikto/Nuclei/SQLMap result means those checks did not produce supported findings under the executed conditions. It does not prove the application has no vulnerabilities. Authentication state, business logic, API paths, and client-side behavior may require separate authorized testing.

11.1 Windows 11 / workstation

Common externally visible services include RPC 135, SMB 445, RDP 3389, WinRM 5985/5986, local web management, database clients/agents, and third-party software. A default endpoint firewall can filter nearly all probes; that is a meaningful posture outcome.

Standard Windows endpoint review

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.40.20.88/32 --interface eth0
--client-name "Acme-Win11" --assessor "your name"
```

11.2 Windows file server

Situation: A file server at 10.40.30.20 is approved. SMB exposure is expected; the objective is to document service posture and correlate only reliable version evidence.

Assessment

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile deep --target 10.40.30.20 --interface eth0 --client-name
"Acme-FileServer" --assessor "your name"
```

Expect: 445/tcp may be open by design. Do not label SMB itself a vulnerability. Findings should be based on specific protocol/version/security evidence, not service presence alone.

11.3 Windows Server / Active Directory domain controller

Typical port	Service context	Interpretation
53	DNS	Expected on many domain controllers; review exposure and version context.
88	Kerberos	Expected AD authentication service.
135	RPC endpoint mapper	Expected Windows infrastructure exposure.

Typical port	Service context	Interpretation
389 / 636	LDAP / LDAPS	Expected directory service; TLS and configuration context matter.
445	SMB	Expected for many AD operations; presence alone is not a finding.
464	Kerberos password change	Common AD service.
3268 / 3269	Global Catalog / GC TLS	Expected on GC domain controllers.
3389	RDP	Administrative exposure; review whether network reachability matches policy.
5985 / 5986	WinRM	Remote management exposure; HTTPS vs HTTP and segmentation matter.

Domain controller assessment - single authorized host

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.40.10.10/32 --interface eth0
--client-name "Acme-AD" --assessor "your name"
```



CAUTION: Directory services are sensitive infrastructure

Use maintenance-window awareness and exact scope. ClientGuard service detection is appropriate; credential guessing, password spraying, exploit execution, or disruptive authentication tests are outside this manual and should not be implied by an open AD port.

11.4 Windows Server with IIS

If service detection finds IIS/HTTP/S, run the server profile for network posture and, when separately approved, a web profile against the exact site URL for application/TLS checks. One server can legitimately have both a host assessment and a URL assessment.

12.1 Linux web server

Host-level assessment

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.40.30.25/32 --interface eth0
--client-name "Acme-LinuxWeb" --assessor "your name"
```

Application-level assessment

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile web --url https://intranet.example.com --client-name
"Acme-LinuxWeb" --assessor "your name"
```

The first command evaluates the host services (for example SSH and HTTPS). The second focuses on the authorized website. Keep host findings and application findings distinct when reporting remediation ownership.

12.2 Linux database server

Common database ports include 3306 MySQL/MariaDB, 5432 PostgreSQL, 6379 Redis, 9200 Elasticsearch, and vendor-specific ports. Exposure to the assessment VLAN may be intentional or may represent segmentation weakness. ClientGuard should document reachability and reliable product evidence without logging into the database unless the methodology separately authorizes authenticated checks.

Deep single-host review

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile deep --target 10.40.30.40 --interface eth0 --client-name
"Acme-DB" --assessor "your name"
```

12.3 Backported package versions

Linux distributions often backport security fixes without changing the upstream-looking version string in the way a simple scanner expects. Treat version-based CVE correlation as a candidate until distribution advisories or package metadata confirm applicability. This is a common reason to use analyst dispositions such as False Positive with a documented vendor backport reason.

13.1 Router profile

Router audit

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py  
--profile router --target 10.40.20.1 --interface eth0 --client-name  
"Acme-Gateway" --assessor "your name"
```

Typical evidence includes reachable management services, HTTP/S posture, SSH/Telnet exposure, TLS certificates/protocols, and identifiable software versions. A management port reachable from an inappropriate VLAN may be a segmentation issue even when the software itself has no CVE match.

13.2 Firewall / security appliance

Treat security appliances like critical servers: use a /32, avoid broad intrusive scans during production hours, and distinguish administrative exposure from product vulnerability. If the management UI is HTTPS, a separate web profile can document TLS/web posture if authorized.

13.3 Printers, cameras, OT/IoT



CAUTION: Fragile devices

Embedded/OT devices may react poorly to aggressive service probes. Use quick/standard profiles only when the device class and test method are explicitly approved. Exclude life-safety and process-control systems unless the engagement plan specifically covers them.

Mixed client network scenario - start to finish

Scenario: A small business authorizes a security review of one office VLAN, one server VLAN, a gateway, and a public portal. A legacy controller is excluded. The goal is to create defensible reports without exploitation.

Asset / scope	Method	Why
10.40.20.0/24 office VLAN	Quick or Standard, exclude sensitive devices	Inventory endpoint/service exposure.
10.40.30.0/24 server VLAN	Standard/Full with exact exclusions	More detailed server posture and intelligence.
10.40.20.1 gateway	Router profile /32	Focus management exposure and TLS/web posture.
https://portal.example.com	Web profile	Application/TLS-specific evidence.
10.40.30.50 legacy controller	Excluded	Written client prohibition / fragility.

14.1 Preflight

Commands

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py
--version
8.0.0
$ python3 /home/kali/ClientGuard/current/clientguard.py
--v8-self-test
{... "self_test":"PASS" ...}
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--precheck
```

14.2 Office VLAN

Command

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.40.20.0/24 --interface eth0 --client-name
"Acme-Office" --assessor "your name"
```


14.3 Server VLAN with exclusion

Command

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile full --target 10.40.30.0/24 --exclude 10.40.30.50/32
--interface eth0 --client-name "Acme-Servers" --assessor "your name"
```

14.4 Gateway

Command

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile router --target 10.40.20.1 --interface eth0 --client-name
"Acme-Gateway" --assessor "your name"
```

14.5 Public portal

Command

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile web --url https://portal.example.com --client-name
"Acme-Portal" --assessor "your name"
```

14.6 Review the outputs

1. Open each report.html and read the Executive Summary and scope first.
2. Check every high/critical item for evidence confidence and whether the service/product evidence is specific enough.
3. Review CVE candidates against vendor/distribution information when backports or appliance versions are possible.
4. Record explicit dispositions rather than deleting inconvenient findings.
5. Preserve the reports and evidence needed for the retest.

14.7 Large enterprise assessments - 300+ endpoints

ClientGuard can be used on an authorized environment with hundreds of reachable endpoints, but the professional approach is **discover broadly, assess in controlled batches, validate selectively, and preserve the scope for every run**. A server does not automatically define the network boundary: the operator must identify the routed subnet(s) and VLANs that are explicitly authorized.

/24

256 total addresses; typically 254 usable host addresses.

/23

512 total addresses; typically 510 usable host addresses.

/22

1,024 total addresses; typically 1,022 usable host addresses.

/16

65,536 total addresses. Do not treat this as one routine monolithic scan without a deliberate plan.



CAUTION: Scale does not expand authorization

A route such as 10.40.0.0/16 may be technically reachable while only two /24 VLANs are approved. Scan only the written scope. Use `--scope-file` and `--exclude` where they make the authorization boundary clearer.

Recommended enterprise workflow

1 Confirm routes and scope

Record interface, routed ranges, VLANs, exclusions, assessment window, and escalation contact.

2 Quick inventory pass

Use the Quick profile to identify reachable hosts and exposed services across the approved range.

3 Split into batches

Run Standard assessments by /24, VLAN, business unit, or another approved logical boundary.

4 Prioritize critical systems

Review servers, externally exposed services, management interfaces, RDP/SMB/SSH, databases, and web applications first.

5 Selective deeper validation

Use Full/Deep or finding-specific safe validation only where the evidence and authorization justify it.

6 Report and retest

Preserve reports, dispositions, remediation evidence, and retest results. Not detected does not mean fixed.

Example: one authorized /23 with more than 300 nodes

Assume the assessment workstation is connected to an authorized corporate segment and the written scope is 10.40.20.0/23. The first step is to verify what Kali can route to - not to guess from a server IP.

MOCK TERMINAL - ROUTE AND SCOPE CHECK

```
$ ip addr show eth0
2: eth0: <UP,LOWER_UP> mtu 1500
    inet 10.40.20.15/23 brd 10.40.21.255 scope global eth0

$ ip route
10.40.20.0/23 dev eth0 proto kernel scope link src 10.40.20.15

# Written authorization confirms the full 10.40.20.0/23 range.
```

Start with a broad inventory-oriented pass:

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py \
--profile quick \
--target 10.40.20.0/23 \
--interface eth0 \
--client-name "Example Enterprise" \
--assessor "your name"

Confirm you are authorized to assess this target and run now? [y/N]: y

[Mocked example - counts will vary]
Preflight           COMPLETE
Host Discovery      COMPLETE
Service Detection   COMPLETE
Report Generation   COMPLETE
Overall             100%
Illustrative result: 327 reachable hosts were observed inside the authorized range.
```

The number **327** above is only a mocked example. A real run can see fewer systems because endpoints may be asleep, firewalled, isolated by ACLs, or on another VLAN. A non-responsive IP is not proof that no device exists.

Batch the vulnerability assessment

After inventory, split the /23 into two /24 Standard assessments. This makes scheduling, troubleshooting, report review, and network-impact control easier than immediately applying a deeper profile to every address.

MOCK TERMINAL - STANDARD BATCH 1

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py \
--profile standard \
--target 10.40.20.0/24 \
--interface eth0 \
--client-name "Example Enterprise - Batch 1" \
--assessor "your name"
```

MOCK TERMINAL - STANDARD BATCH 2

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py \
--profile standard \
--target 10.40.21.0/24 \
--interface eth0 \
--client-name "Example Enterprise - Batch 2" \
--assessor "your name"
```

Example: multiple VLANs and explicit exclusions

Large environments are commonly separated into workstation, server, wireless, printer/IoT, and management VLANs. Do not replace those exact ranges with a larger supernet unless that supernet itself is authorized.

enterprise_scope.txt - illustrative approved scope:

```
10.50.10.0/24    # employee workstations
10.50.20.0/24    # application servers
10.50.30.0/24    # corporate Wi-Fi
10.50.40.0/24    # printers / IoT - only if specifically authorized
```

Run a Quick inventory while excluding a sensitive host that the client has placed out of scope:

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py \
--profile quick \
--scope-file enterprise_scope.txt \
--interface eth0 \
--exclude 10.50.40.50/32 \
--client-name "Example Enterprise" \
--assessor "your name"
```

What to expect with 300+ endpoints

- **Longer run time:** filtered ports, slow devices, WAN latency, service probes, and profile depth all increase duration.
- **More network telemetry:** firewalls, IDS/IPS, EDR, and SOC tools may generate alerts during an approved assessment. Coordinate the window.
- **Different visibility by VLAN:** routing and ACLs determine what the Kali assessment host can reach. One interface does not guarantee visibility into every subnet.
- **Prioritize servers:** after inventory, use /32 assessments for high-value systems when you need tighter evidence and easier review.
- **Avoid Deep everywhere by default:** use deeper checks selectively rather than turning the broad discovery phase into the most intensive assessment.
- **Preserve exclusions:** fragile legacy systems, OT/life-safety devices, or client-prohibited assets should remain explicitly out of scope unless separately approved.
- **Expect incomplete discovery:** sleeping devices, host firewalls, endpoint isolation, or NAC can hide systems from a network-based scan.
- **Do not infer remediation:** a host or service that disappears on retest is not automatically fixed; investigate and record the analyst disposition.



Enterprise rule of thumb

Use Quick to understand the authorized estate, Standard in manageable subnet/VLAN batches, and Full/Deep or finding-specific validation on selected systems. ClientGuard still does not automatically exploit, remediate, close findings, accept risk, expand scope, or persist credentials.

15.1 Active discovery authorization file

authorized_discovery.json

CLIENTGUARD TERMINAL

```
{
  "assessment_id": "acme-server-001",
  "analyst": "your name",
  "authorized": true,
  "scope": {
    "authorized_targets": ["10.40.30.20/32"],
    "exclusions": [],
    "assessment_zone": "CLIENT-SERVER-VLAN"
  },
  "requests": [
    {"kind": "nmap", "target": "10.40.30.20"}
  ]
}
```

Run safe active discovery

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--v8-active-discovery-json authorized_discovery.json
```

Expect a JSON receipt with `explicit_authorization true`, `scope_enforced true`, `production_write false`, `exploit_execution false`, `result_count`, `request_count`, and the safe-discovery tool output. An unauthorized file should fail closed rather than silently scan.

15.2 Controlled workflow file

workflow.json - evidence processing example

CLIENTGUARD TERMINAL

```
{
  "assessment_id": "acme-server-001",
  "analyst": "your name",
  "authorized": true,
  "scope": {
    "authorized_targets": ["10.40.30.20/32"],
    "exclusions": [],
    "assessment_zone": "CLIENT-SERVER-VLAN"
  },
  "scanner_results": {
    "nmap": {
      "target": "10.40.30.20",
      "observed_service": {
        "port": 445,
        "protocol": "tcp",
        "state": "open",
        "service": "microsoft-ds"
      }
    }
  },
  "metadata": {"environment": "Authorized client server VLAN"}
}
```

Process controlled workflow

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py
--v8-workflow-json workflow.json
```

The controlled workflow processes supplied evidence through v8 stages. It is useful for structured automation or when scanner evidence has already been collected under an explicit authorization receipt.

15.3 Native integrity commands

Command	Purpose
--v8-runtime-info	Show v8 runtime/compatibility identity.
--v8-legacy-integrity	Verify the exact embedded v7.12.0 compatibility parent.
--deployment-info	Show deployment lineage and frozen model/schema identity.
--v8-capabilities	Show external scanner availability.
--v8-self-test	Run local v8 integrity/safety self-test.

What each external tool does

Tool	Role inside / around ClientGuard	Typical safe evidence	Not the same as
Nmap	Host/service discovery and conservative version detection.	Open/closed/filtered ports, service names, version hints, CPE candidates, safe NSE where configured.	Exploit proof.
Nikto	Web-server hygiene scanner.	Server headers, common risky files/configurations, outdated web-server indicators.	Authenticated application business-logic testing.
Nuclei	Template-based web/service validation.	Known pattern/template matches with reproducible request/response evidence.	Automatic vulnerability truth; template context still matters.
SQLMap	SQL-injection testing utility used conservatively through ClientGuard.	Parameterized URL behavior and possible SQLi indicators.	Permission to dump databases, obtain shells, or write files.
OpenSSL	TLS handshake/certificate/protocol inspection.	Certificate chain, subject/SAN, cipher/protocol behavior.	Full web application testing.
curl	HTTP reachability and header/request diagnostics.	Status code, headers, redirect behavior, simple API reachability.	Vulnerability scanner.
ip / ip route	Linux interface and routing diagnostics.	Addresses, UP/DOWN state, source route, gateway/subnet context.	Network scanner.
nmcli / iw	Wireless connection/interface status on Linux.	Association, interface state, connection name, address context.	Wi-Fi cracking or monitor-mode attack tools.
sha256sum	File integrity verification.	Exact cryptographic digest match.	Code-signing identity or malware analysis.

16.1 Nmap flags seen in ClientGuard-style safe discovery

Flag	Meaning	Why it matters
-sT	TCP connect scan.	Uses normal TCP connect behavior; does not require raw SYN scanning semantics.
-Pn	Treat target as up; skip ICMP host-discovery dependence.	Useful when Windows/firewalls block ping but the host is reachable.
--version-light	Lightweight service version detection.	Balances identification with scan time and probe volume.
--reason	Show why Nmap assigned a state.	Helps distinguish response vs no-response/filtered behavior.
-T3	Normal/moderate timing template.	Conservative default compared with aggressive timing.
--top-ports 100	Check Nmap top 100 common ports.	Faster safe discovery than all 65,535 ports.
-sV	Service/version detection (manual diagnostic examples).	Provides product/version evidence when a deeper manual check is needed.

Command-line option reference - common --* flags

The installed runtime help is authoritative for exact syntax:

Show compatibility-runtime help

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py --help
```

Option	What it does	What to expect
--help	Show available delegated/compatibility options and examples.	Expect usage/examples; no assessment.
--version	Show v8 release version.	Expect 8.0.0.
--assistant	Open the interactive ClientGuard Command Assistant.	Expect menu prompts.
--profile quick	Fast network audit profile.	Expect reduced-depth network/service assessment.
--profile standard	Balanced default client audit.	Expect full normal progress/reporting pipeline.
--profile full	Comprehensive authorized assessment profile.	Expect longer/deeper workflow than Standard.
--profile deep	Deep host-focused assessment profile.	Expect longer runtime; best for approved single-host depth.
--profile router	Network gateway/appliance profile.	Expect router/service/web/TLS posture focus.
--profile web	Website/URL profile.	Use with --url; expect web/TLS oriented checks.
--target TARGET	Define an IP, host, or CIDR target for network profiles.	Scope must match written authorization.
--url URL	Define an authorized web URL for web-oriented operations.	Use full scheme such as https://.
--interface IFACE	Bind network assessment to eth0, wlan0, etc.	Choose the routed interface that reaches target.

Option	What it does	What to expect
--client-name NAME	Label report hierarchy/client identity.	Expect client-specific report folder/metadata.
--assessor NAME	Record operator/assessor identity.	Expect report/evidence attribution.
--scope-file FILE	Load authorized scope from a file.	Review every range before execution.
--exclude CIDR	Exclude an address/range from broader scope.	Expect excluded host not to be assessed.
--precheck	Run environment/tool readiness checks.	No client scan.
--install-tools	Install supported dependency set.	Use sudo and workstation change approval.
--dry-run	Preview an operation such as tool installation without applying it.	Expect plan/output only.
--include-optional	Include optional tools during installation.	Only add approved methodology dependencies.
--dashboard	Launch/use the local dashboard function.	May require companion --open-browser.
--open-browser	Open the dashboard in the local browser.	Expect local browser launch.
--baseline-save	Save the current assessment as a baseline.	Expect baseline metadata/artifact.
--baseline-name NAME	Give a saved baseline a human label.	Use with baseline-save.
--baseline-from FILE	Compare/operate relative to an earlier baseline.	Expect baseline comparison context.
--retest-from FILE	Run/prepare retest comparison from a previous report.	Expect FIXED/STILL_PRESENT/CHANGED/REGRESSED/NEW/NOT_RETESTED style outcomes where supported.
--trend-from FILE	Compare trend against an older report.	Expect historical comparison context.
--trend-dir DIR	Use a report directory as trend history.	Expect multi-run trend context.
--ledger-metadata FILE	Attach explicit remediation-owner/ledger metadata.	Governance metadata; does not remediate automatically.

Option	What it does	What to expect
--governance-metadata FILE	Attach governance/review metadata.	Explicit governance evidence only.
--risk-acceptance-metadata FILE	Attach explicit risk-acceptance metadata.	Accepted risk does not reduce technical exposure.
--validate-finding REPORT ID	Validate a specific prior finding through the supported validation path.	Expect updated evidence/validation result, not automatic closure.
--verify-finding REPORT ID	Verify a specific finding against current evidence.	Expect verification result tied to finding ID.
--safe-validate REPORT ID	Perform safe non-destructive validation of a prior finding.	Expect validation evidence; no exploitation.
--set-disposition REPORT ID VALUE	Set an analyst disposition for a specific finding.	Explicit analyst action is recorded.
--disposition-reason TEXT	Document why a disposition was chosen.	Use with disposition changes for auditability.
--nuclei-validate URL	Run explicit Nuclei URL validation.	Expect template-based evidence on authorized URL.
--sqlmap-check URL	Run conservative SQLMap check.	Use only on explicit authorized parameterized URLs.
--list-checkpoints	List resumable assessment checkpoints.	Expect IDs/status; no scan.
--resume ENGAGEMENT_ID	Resume a supported interrupted engagement/checkpoint.	Expect continuation from stored state.
--generate-completion	Generate shell completion support.	Operator convenience; no scan.
--release-info	Show release/build information.	No scan.
--storage-health	Check managed storage health.	No scan.
--deployment-info	Show deployment/runtime lineage information.	No scan; v8 native command.
--deployment-health	Check installed deployment health.	No scan.
--deployment-backup	Create/use supported deployment backup workflow.	Administrative operation; not assessment data backup.
--install-clientguard	Run supported managed ClientGuard installation workflow.	Administrative; use only for deployment management.

Option	What it does	What to expect
--deploy-dry-run	Preview managed ClientGuard deployment changes.	No production switch expected.
--v8-self-test	Native v8 self-test.	JSON PASS/identity/safety output.
--v8-runtime-info	Native v8 runtime/parent information.	JSON runtime identity.
--v8-capabilities	Native v8 scanner capability inventory.	JSON scanner states.
--v8-active-discovery-json FILE	Run explicitly authorized safe active discovery described in JSON.	Fail closed if authorization/scope invalid.
--v8-workflow-json FILE	Process controlled v8 workflow JSON.	Structured receipt/output.
--v8-legacy-integrity	Verify embedded compatibility parent hash.	Expect verified true and parent SHA.

i NOTE: Why some examples show older filenames

The v8 wrapper delegates non-native arguments to the exact embedded v7.12.0 compatibility runtime. The assistant may display a compatibility filename in a generated-command preview. For installed operation, keep invoking /home/kali/ClientGuard/current/clientguard.py.

18.1 Typical report tree

Example

```
CLIENTGUARD TERMINAL

/home/kali/reports/
├── Acme-Internal/
│   ├── 2026/
│   │   ├── Standard/
│   │   │   ├── 4_Aug242026_1015am/
│   │   │   │   ├── report.html
│   │   │   │   └── report.json
```

18.2 HTML vs JSON

Artifact	Audience	Use
report.html	Human analyst/client	Readable summary, findings, evidence, recommendations.
report.json	Automation/analyst tooling	Machine-readable findings, receipts, scope, service/CVE evidence and lifecycle data.

18.3 Intelligence terms

Term	Meaning	How to use it
CVE	Public vulnerability identifier.	A candidate requires product/version applicability evidence.
CVSS	Technical severity score.	Use as severity context, not as a business-priority substitute.
EPSS	Probability-oriented exploitation likelihood estimate.	Useful for prioritization among applicable CVEs.
KEV	Known Exploited Vulnerabilities catalog signal.	A strong prioritization signal when the CVE is actually applicable.
CPE	Structured product identity string.	Can strengthen mapping from service evidence to CVEs.

Term	Meaning	How to use it
Confidence	How strongly evidence supports the finding.	High confidence comes from specific/reproducible evidence, not severity alone.

18.4 Examples of correct interpretation

Observed result	Correct interpretation	Incorrect interpretation
3389/tcp open ms-wbt-server	RDP is reachable from this assessment path. Review whether exposure is intended and gather version/policy context.	RDP is vulnerable.
445/tcp open microsoft-ds	SMB is exposed/reachable.	The host has EternalBlue.
CVE candidate from version evidence	Potentially applicable CVE requiring applicability review/validation.	Confirmed exploitability.
0 CVE matches	No supported CVE mapping from current evidence.	The host has no vulnerabilities.
Filtered ports	Firewall/network policy did not return probe responses.	ClientGuard failed.

19.1 Retest a previous client

Assistant path

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--assistant
Select: 6 # Retest / Compare Previous Client
```

Direct example

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile full --target 10.40.20.15 --retest-from latest.json
--interface eth0 --client-name "Acme-Retest" --assessor "your name"
```

Outcome	Meaning
FIXED	Prior evidence is no longer present under the retest conditions and supported remediation evidence exists according to workflow semantics.
STILL_PRESENT	The same issue/evidence remains.
CHANGED	Related condition changed materially; analyst review required.
REGRESSED	Previously improved condition has returned/worsened.
NEW	Finding appears in retest but not baseline.
NOT_RETESTED	No valid retest evidence was collected for that item.



CAUTION: Not detected is not fixed

A closed port, timeout, changed firewall, or unavailable service can make evidence disappear. ClientGuard preserves the distinction between absence of detection and verified remediation.

19.2 Validate or verify one finding

Examples

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py
--validate-finding previous.json CG-8E52C1A9
$ python3 /home/kali/ClientGuard/current/clientguard.py --verify-finding
previous.json CG-8E52C1A9
$ python3 /home/kali/ClientGuard/current/clientguard.py --safe-validate
previous.json CG-8E52C1A9
```

19.3 Set analyst disposition

Example

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py
--set-disposition previous.json CG-8E52C1A9 'False Positive'
--disposition-reason 'Vendor backport confirmed'
```

A disposition records analyst judgment; it should never be used to hide uncertainty. Document the evidence supporting the decision.

20.1 Dashboard

Open dashboard

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py --dashboard  
--open-browser
```

Use the dashboard to review supported stored assessment information. It does not change the authorization boundary for new scans.

20.2 Presets

The Assistant can save a generated command as a preset and later reuse it. Before reusing a preset, re-read the target and exclusions. A preset is convenience, not proof that the scope is still authorized.

20.3 Checkpoints and resume

Commands

CLIENTGUARD TERMINAL

```
$ python3 /home/kali/ClientGuard/current/clientguard.py  
--list-checkpoints  
$ python3 /home/kali/ClientGuard/current/clientguard.py --resume  
ENGAGEMENT_ID
```

Resume only the engagement you intend to continue, and confirm its scope has not changed since interruption.

20.4 Deployment/admin health commands

Command	Purpose
--release-info	Display release information.
--storage-health	Check storage subsystem health.
--deployment-info	Display current v8 deployment identity/lineage.
--deployment-health	Check managed deployment health.

Command	Purpose
--deployment-backup	Use supported deployment backup operation.
--install-clientguard --deploy-dry-run	Preview managed installation/update changes.
--install-clientguard	Perform supported managed installation/update workflow when intentionally administering ClientGuard.

Troubleshooting and common operator mistakes

Symptom	Likely meaning	Safe response
No menu when script is run with no arguments	v8 no-argument behavior shows native commands.	Run <code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --assistant</code> .
Ping fails but target exists	Host firewall may block ICMP.	Check route; use approved ClientGuard scan or diagnostic <code>nmap -Pn</code> . Do not disable firewall merely for scanning.
Everything is filtered	Firewall/ACL is dropping probes.	Treat as posture evidence. Verify assessment path and scope.
Wrong interface selected	Docker/VPN/loopback route does not reach target.	Use <code>ip -br addr</code> and <code>ip route get TARGET</code> before launch.
No CVE matches	Service banner/version/CPE may be generic.	Do not force a CVE. Review exact service evidence.
Nmap service unrecognized	Service responded but fingerprint was not specific.	Record exposure; manually review only if methodology requires it.
Report path not found under current directory	Saved path is relative to configured report root.	Use <code>find /home/kali/reports -type d -name RUN_NAME</code> or copy the printed full path if shown.
Permission denied deleting reports	Some assessment files were created with <code>sudo/root</code> ownership.	Use narrowly scoped <code>sudo find PATH -mindepth 1 -delete</code> only after preserving required reports.
Scanner unavailable in --v8-capabilities	Dependency missing/not reachable.	Run <code>--precheck</code> ; review <code>--install-tools --dry-run</code> before installing.
Website scan fails early	DNS/TLS/network issue or URL not reachable.	Use <code>curl -I</code> and <code>openssl s_client</code> to distinguish reachability/TLS from scanner logic.
Assistant suggests entire /24	It derives local subnet as a convenience.	Replace it with written scope, often /32 for one host.
Retest says not detected	Target/service may be unavailable or filtered.	Do not call it fixed without remediation evidence.

21.1 Verify runtime integrity when behavior is unexpected

Commands

```
$ sha256sum /home/kali/ClientGuard/current/clientguard.py
$ python3 /home/kali/ClientGuard/current/clientguard.py
--v8-self-test
$ python3 /home/kali/ClientGuard/current/clientguard.py
--v8-legacy-integrity
```

22.1 Before every real assessment

- Written authorization and scope are available.
- Client name and assessor identity are correct.
- Targets/ranges and exclusions are reviewed.
- Correct interface and route are confirmed.
- Maintenance-window or fragile-device restrictions are understood.
- ClientGuard version/self-test/precheck are healthy.
- Required tools are available; optional tools are approved.
- Report destination has sufficient space and is protected.

22.2 Before pressing y in the Assistant

- Read the generated command character by character.
- Confirm --target or --url is correct.
- Confirm --interface is correct.
- Confirm exclusions are present.
- Confirm client-name/assessor labels are correct.
- Confirm profile depth is appropriate.

22.3 After assessment

- Wait for Complete | Overall 100% or document the failure state.
- Record the Saved path.
- Open report.html and inspect report.json as needed.
- Review high/critical findings for evidence confidence and applicability.
- Do not equate open ports with vulnerabilities.
- Preserve evidence for retest.
- Use explicit dispositions rather than editing findings out of reports.

22.4 Mock terminal - complete single Windows Server Standard scan

Example session

```
$ ip -br addr show eth0
eth0 UP 10.40.20.44/24
$ ip route get 10.40.20.15
10.40.20.15 dev eth0 src 10.40.20.44
$ python3 /home/kali/ClientGuard/current/clientguard.py --version
8.0.0
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile
standard --target 10.40.20.15/32 --interface eth0 --client-name
"Acme-WinServer" --assessor "your name"
Confirm you are authorized to assess this target and run now? [y/N]:
y
Preflight COMPLETE
Service Detection COMPLETE
CVE / EPSS / KEV COMPLETE
Safe Validation COMPLETE
Report Generation COMPLETE
Complete COMPLETE | Overall 100.00%
Saved: Standard/7_Aug242026_1130am
report.html - report.json
```

22.5 Mock terminal - web assessment

Example session

```
$ curl -I https://portal.example.com
HTTP/2 200
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile
web --url https://portal.example.com --client-name "Acme-Portal"
--assessor "your name"
Confirm authorization? [y/N]: y
Web/TLS checks COMPLETE
Safe validation COMPLETE
Report Generation COMPLETE
Complete 100.00%
```

Apple/macOS endpoints, Mac servers, and other Apple devices

ClientGuard can assess Apple computers from the normal Kali operator workstation without installing ClientGuard on the Mac. The assessment is network/service driven: ClientGuard evaluates what an explicitly authorized Mac exposes over the assessment path, correlates reliable product/version evidence, performs supported safe validation, and produces the same evidence/reporting workflow used for Windows, Linux, routers, and other network targets.

23.1 Apple systems that fit ClientGuard's network-assessment model

Apple target	Typical role	How ClientGuard is used	Important limitation
MacBook / MacBook Pro / MacBook Air	Employee or administrator endpoint	Standard or Quick profile against the authorized /32 address; use eth0 or wlan0 according to the Kali route.	Unauthenticated network assessment does not prove FileVault, Gatekeeper, SIP, TCC or MDM compliance.
iMac / Mac Studio / Mac Pro	Desktop, workstation, creative/engineering system	Standard/Full profile for reachable services; Web profile for an explicitly authorized local web service.	Do not infer local patch state merely from an open port.
Mac mini used as a server	File server, build host, web host, management utility	Standard/Full/Deep host review; add Web profile for an approved HTTP/S URL.	Treat the host assessment and web-application assessment as separate evidence domains.
Apple Silicon or Intel Mac	Any supported Mac hardware architecture	Same network workflow and scope rules.	CPU architecture alone does not establish software vulnerability applicability.
iPhone / iPad / Apple TV	Mobile/embedded Apple device	Only limited reachable-service coverage when explicitly authorized.	ClientGuard is not a jailbreak, MDM bypass, mobile-forensics or local configuration-audit tool.

23.2 Common macOS services and what they mean

A Mac can be quiet on the network or expose several legitimate services depending on System Settings, enterprise management, installed software, and its role. Service presence is not itself a vulnerability. ClientGuard should report reachability and reliable evidence, then correlate only when the product/version information is specific enough.

Port / protocol	Common Apple/macOS use	Operator interpretation
22/tcp	SSH / Remote Login	Open means remote shell service is reachable. Review product/version evidence and scope; do not attempt passwords.
445/tcp	SMB file sharing	May be intentional for shared folders or enterprise interoperability. SMB presence alone is not a finding.
5900/tcp	Screen Sharing / VNC	Reachable remote desktop exposure. Report posture and validation evidence; do not authenticate unless separately authorized.
80/443 tcp	Local web service, management UI, development app	Use host profile for service posture and Web profile for the exact approved URL when application testing is in scope.
631/tcp	IPP / printing	May identify printing services. Treat version/CPE evidence conservatively.
548/tcp	AFP on older/legacy Mac environments	Legacy exposure may justify review, but service presence alone does not prove vulnerability.
3283/tcp	Apple Remote Desktop related service in some environments	Management exposure may be expected; document reachability and specific evidence.
5353/udp	Bonjour / mDNS discovery context	UDP discovery is methodology-dependent. A default TCP-focused ClientGuard run may not enumerate this; do not claim absence from a TCP-only scan.

23.3 Preflight an authorized Mac from Kali

Example scenario: the client authorizes one corporate Mac at 10.55.8.60. Kali is connected to the approved assessment VLAN on eth0. Check the source interface and route before launching ClientGuard.

```
$ ip -br addr show eth0
eth0 UP 10.55.8.42/24
$ ip route get 10.55.8.60
10.55.8.60 dev eth0 src 10.55.8.42
$ python3 /home/kali/ClientGuard/current/clientguard.py --version
8.0.0
```

If ICMP ping is filtered, do not assume the Mac is offline. macOS firewall policy or upstream ACLs may drop echo requests. Use the authorized ClientGuard assessment path and record filtered/no-response behavior as posture evidence.

23.4 Assistant/menu-driven Mac assessment

The Assistant workflow is the easiest field method when you want ClientGuard to build the command interactively. The operator still owns the authorization decision and must replace any convenient subnet default with the written scope.

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--assistant
CLIENTGUARD COMMAND ASSISTANT
1) Standard Client Audit
Select: 1
Select interface: eth0
Client/company name: Acme-MacFleet
Assessor: your name
Authorized scope: 10.55.8.60/32
Choose: 1 # Run now
Confirm you are authorized to assess this target and run now? [y/N]:
y
```

Expected behavior: preflight, authorization, service detection, conservative CVE/EPSS/KEV enrichment when reliable product evidence exists, supported safe validation, report generation, and a saved HTML/JSON result. An open SSH, SMB, Screen Sharing or web service remains an exposure observation until evidence supports a more specific finding.

23.5 Direct CLI - one Mac endpoint

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile standard --target 10.55.8.60/32 --interface eth0 --client-name
"Acme-MacBook" --assessor "your name"
```

Use /32 for a single Mac. If the Mac is reached over an authorized Wi-Fi connection, use the interface that actually owns the route (for example wlan0) rather than forcing eth0.

23.6 Mac mini or Mac workstation acting as a server

Example: a Mac mini at 10.55.8.70 hosts SSH, SMB and an internal HTTPS application. Run the host assessment first, then run a URL-oriented Web profile only if the website/application is separately in scope.

CLIENTGUARD TERMINAL

```
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py
--profile full --target 10.55.8.70/32 --interface eth0 --client-name
"Acme-MacServer" --assessor "your name"
$ sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile
web --url https://mac-mini.internal.example --client-name
"Acme-MacServer-Web" --assessor "your name"
```

Keep host-level and application-level findings separate. For example, TCP/443 exposure belongs to the host/service picture; TLS/header/application observations from the exact URL belong to the web assessment.

23.7 Apple Silicon versus Intel Macs

ClientGuard's network-assessment method is the same for Apple Silicon and Intel Macs because the assessment is driven by reachable services and product evidence. Architecture can matter when deciding whether a specific advisory applies, but it must come from reliable evidence or vendor documentation. Do not assign or dismiss a CVE merely because the target is an M-series or Intel Mac.

23.8 What ClientGuard cannot prove from an unauthenticated Mac network scan

- FileVault disk-encryption status or recovery-key escrow.
- Gatekeeper configuration, notarization policy, or local application trust decisions.
- System Integrity Protection (SIP) state.
- TCC/privacy permissions, local admin membership, password policy, or secure-token state.
- MDM enrollment/compliance, configuration profiles, endpoint-security agent health, or local firewall policy beyond externally observed behavior.
- Complete installed-application inventory or exact macOS patch level when the network service evidence does not expose it.

Those questions require an authenticated/local configuration audit, MDM/EDR evidence, or a separately approved endpoint-assessment method. ClientGuard should not fabricate local compliance from network observations.

23.9 iPhone, iPad, Apple TV, and similar Apple devices

ClientGuard can only assess services that are actually reachable over the authorized network path. Many mobile Apple devices expose little or nothing to a normal TCP assessment, which may legitimately produce mostly filtered/closed results. Treat that as observed network posture, not proof that the device is fully secure. ClientGuard does not bypass MDM, unlock devices, jailbreak them, extract local data, or perform mobile forensics.

23.10 Real-world Mac reporting example

CLIENTGUARD TERMINAL

```
MOCKED TERMINAL / REPORT INTERPRETATION
Target: 10.55.8.60/32 (authorized corporate Mac)
22/tcp open ssh
445/tcp open microsoft-ds
5900/tcp filtered vnc
CVE correlation: no reliable match for generic banners
Interpretation: SSH and SMB are reachable by design; Screen Sharing is
filtered.
Do not write: "Mac is vulnerable because SSH is open."
Write: "SSH is reachable from the assessment VLAN; no specific
exploitable condition was confirmed from the available service
evidence."
```

Do not enable Remote Login, Screen Sharing, File Sharing, or other services on a real client Mac merely to make a scan produce results. Assess the posture that exists during the authorized window. If a service is expected but filtered, record that outcome and confirm intended configuration with the system owner.

Installed path reference

Purpose	Path
Managed executable	/home/kali/ClientGuard/current/clientguard.py
Release copy	/home/kali/ClientGuard/releases/8.0.0/clientguard.py
Stable release seal	/home/kali/ClientGuard/releases/8.0.0/ STABLE_RELEASE_SEAL.json
Rollback runtime	/home/kali/ClientGuard/rollback/clientguard_before_8.0.0.py
Release archive	/home/kali/ClientGuard_release_archive/v8.0.0/
Archive manifest	/home/kali/ClientGuard_release_archive/v8.0.0/ ARCHIVE_SHA256SUMS
Common report root	/home/kali/reports/

Verify archived release evidence

CLIENTGUARD TERMINAL

```
$ cd /home/kali/ClientGuard_release_archive/v8.0.0
$ sha256sum -c ARCHIVE_SHA256SUMS
... OK
$ sha256sum ARCHIVE_SHA256SUMS
0589aff2720f35d6f4b867a018023806b5e7b9da10e87093fdcbb86e825b7de3
ARCHIVE_SHA256SUMS
```

OS / server coverage matrix

Target	Can assess?	Suggested mode	Notes
Windows 10/11	Yes - network target	Standard/Quick	RDP, SMB, RPC, WinRM, web/third-party services. Endpoint firewall often filters probes.
Windows Server	Yes - network target	Standard/Full/Deep	AD, DNS, LDAP, Kerberos, SMB, RDP, WinRM, IIS, databases. Use critical-infrastructure caution.
Linux server	Yes - network target	Standard/Full/Deep	SSH, web, DNS, mail, DB, NFS/RPC, custom daemons. Consider vendor backports.
Web application	Yes - URL target	Web	HTTP/S, TLS, Nikto, Nuclei, conservative SQLMap checks where explicitly authorized.
Router/firewall	Yes - network target	Router	Management exposure, web/TLS, SSH/Telnet/ service versions. No password attacks.
Connected Wi-Fi LAN	Yes - via wlan0	Quick/Standard/Wi-Fi Assistant	IP-layer assessment after legitimate association. No deauth/cracking.
macOS (MacBook/iMac/Mac mini/Mac Studio/Mac Pro)	Yes - network/service target	Standard/Full; Web for URLs	SSH, SMB, Screen Sharing/VNC, web/TLS, printing and third-party services when reachable. Apple Silicon vs Intel does not change the network-assessment method. Local FileVault/Gatekeeper/SIP/MDM posture is outside unauthenticated network scope.
Cloud VM with reachable IP	Yes if explicitly authorized and provider rules permit	Standard/Web	Treat like server/web target. Cloud provider policy and scope matter.

Target	Can assess?	Suggested mode	Notes
Container service / Kubernetes endpoint	Reachable service coverage	Standard/Web	Assesses exposed service endpoints; not a full cluster configuration audit.
OT/IoT embedded	Possible but caution	Quick/Standard only if approved	Fragile devices may not tolerate probing; written methodology required.
iPhone/iPad/Apple TV	Limited reachable-service coverage	Quick/Standard only when explicitly approved	Only services actually reachable over the authorized network can be assessed. This is not an MDM bypass, jailbreak, local configuration, or mobile forensics audit.

Command recipes

Task	Command
Open assistant	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --assistant</code>
Version	<code>python3 /home/kali/ClientGuard/current/clientguard.py --version</code>
Self-test	<code>python3 /home/kali/ClientGuard/current/clientguard.py --v8-self-test</code>
Capabilities	<code>python3 /home/kali/ClientGuard/current/clientguard.py --v8-capabilities</code>
Precheck	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --precheck</code>
Standard one host	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile standard --target 10.40.20.15/32 --interface eth0 --client-name "Acme" --assessor "your name"</code>
Full subnet with exclusion	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile full --target 10.40.20.0/24 --exclude 10.40.20.50/32 --interface eth0 --client-name "Acme" --assessor "your name"</code>
Quick connected Wi-Fi	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile quick --target 10.30.50.0/24 --interface wlan0 --client-name "Acme-WLAN" --assessor "your name"</code>
Router	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile router --target 10.40.20.1 --interface eth0 --client-name "Acme-Gateway" --assessor "your name"</code>
Website	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile web --url https://portal.example.com --client-name "Acme-Web" --assessor "your name"</code>
Dashboard	<code>python3 /home/kali/ClientGuard/current/clientguard.py --dashboard --open-browser</code>

Task	Command
Active discovery JSON	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --v8-active-discovery-json authorized_discovery.json</code>
Workflow JSON	<code>python3 /home/kali/ClientGuard/current/clientguard.py --v8-workflow-json workflow.json</code>
Standard Mac endpoint	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile standard --target 10.55.8.60/32 --interface eth0 --client-name "Acme-Mac" --assessor "your name"</code>
Mac server host review	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile full --target 10.55.8.70/32 --interface eth0 --client-name "Acme-MacServer" --assessor "your name"</code>
Mac-hosted website	<code>sudo python3 /home/kali/ClientGuard/current/clientguard.py --profile web --url https://mac-mini.internal.example --client-name "Acme-MacServer-Web" --assessor "your name"</code>

END

Final operator reminder

ClientGuard is most valuable when the operator preserves the difference between what the network exposed, what intelligence correlated, what safe validation supported, and what a human analyst ultimately decided. A precise report with zero unsupported CVEs is better than an impressive report full of assumptions.

CLIENTGUARD TERMINAL

```
PASS: Field-ready workflow
Verify runtime -> confirm authorization -> verify
interface/route -> define exact scope/exclusions -> choose
Assistant or direct profile -> review command -> execute ->
inspect evidence/report -> document analyst decisions -> preserve
retest evidence.
```

ClientGuard v8.0.0 - Written by Raid9sec